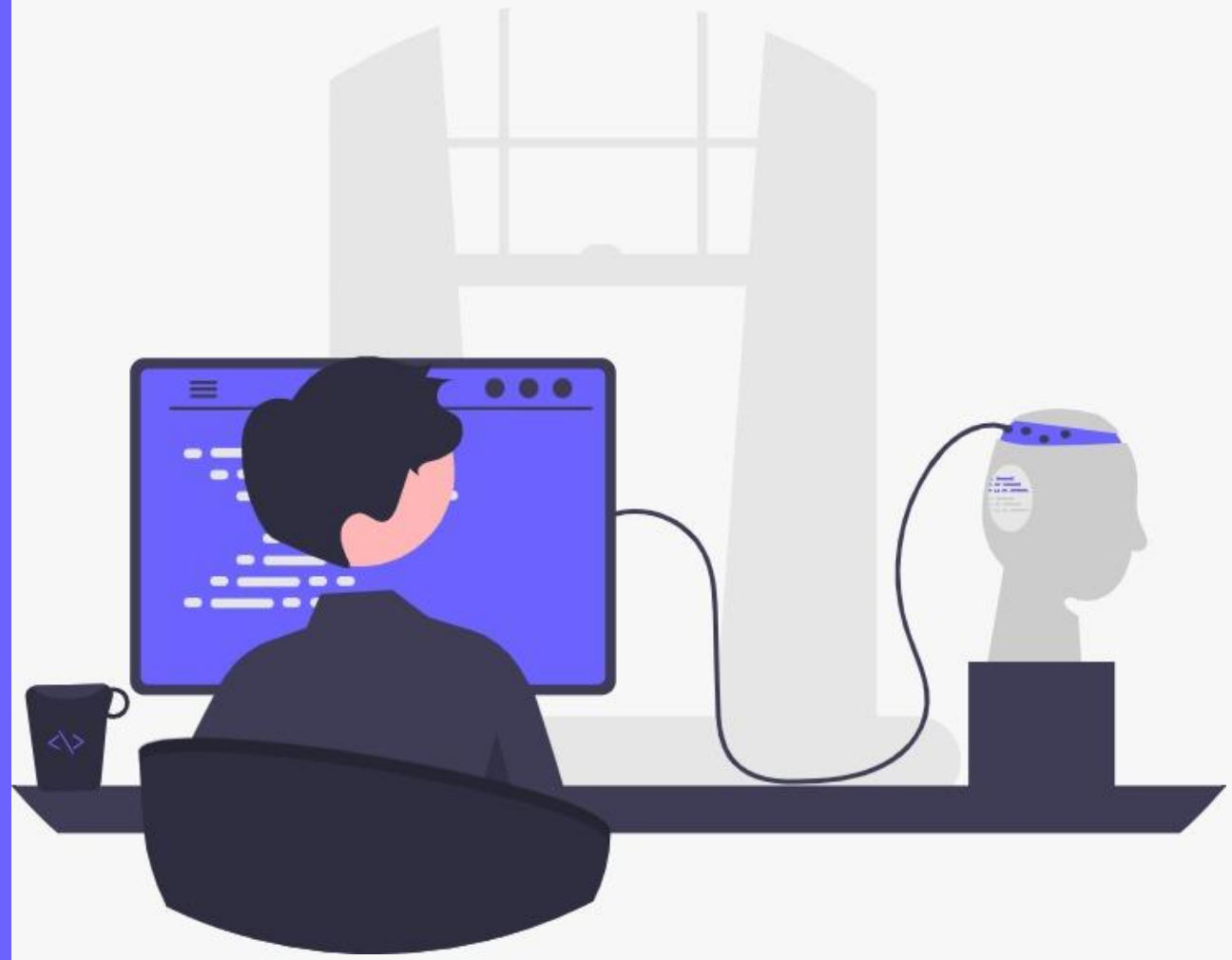


PROJECT WORK ESIB

18 Dicembre 2023

Carmela Amato
Sara Cascetta
Federica Cirillo

CASO STUDIO

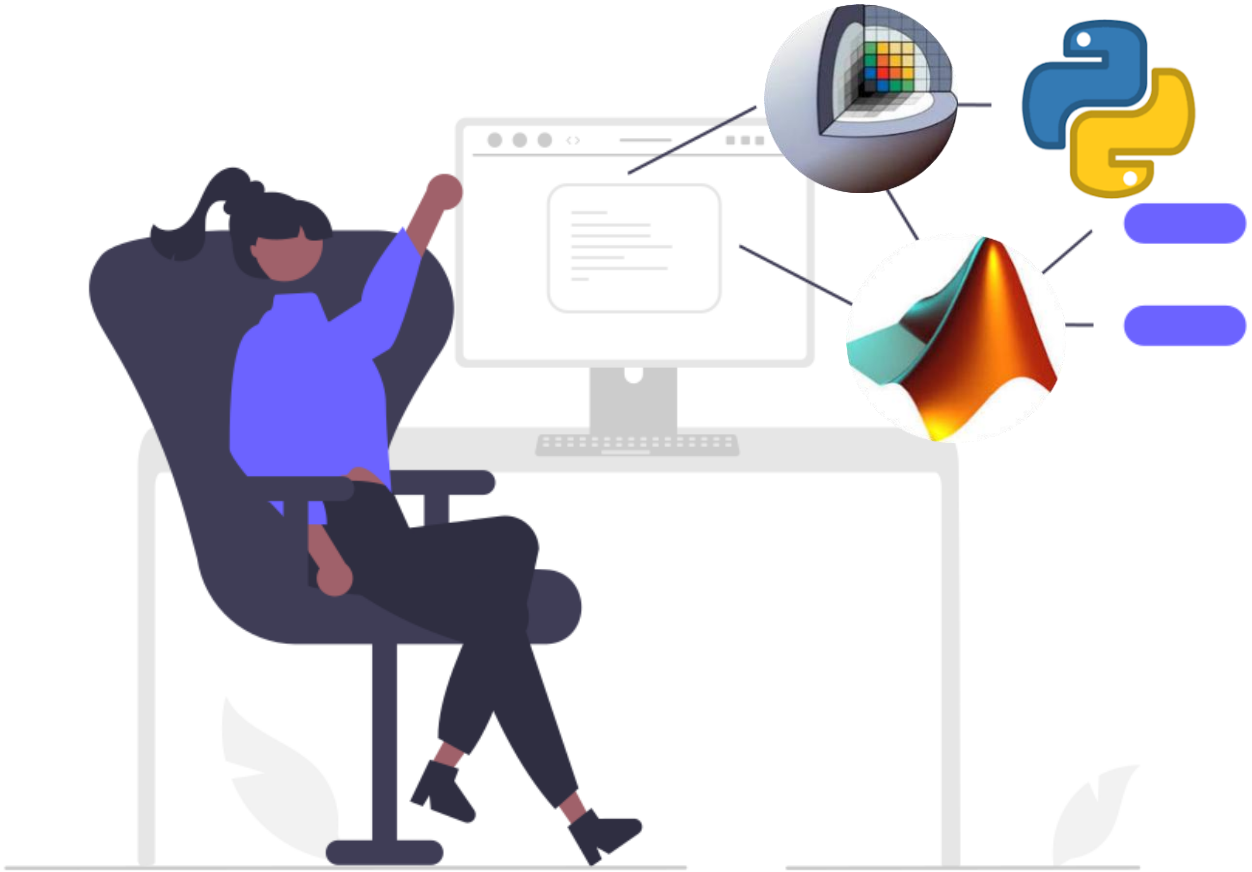
La *Paralisi Sopranucleare Progressiva* (PSP) è una **malattia neurodegenerativa** rara, causata da una perdita progressiva e selettiva di cellule nervose in specifiche regioni del cervello.
Il deterioramento dei neuroni coinvolti è causa di movimenti lenti, rigidità muscolare, incapacità di muovere gli occhi e tendenza a cadere all'indietro.
Sono state descritti **5 fenotipi**: la **sindrome di Richardson** (**PSP-RS**), che è la variante clinica più comune, e altri 4 fenotipi (**vPSP**).

OBIETTIVO

Distinzione di pazienti con fenotipo Richardson (PSP-RS) da pazienti con altri fenotipi (vPSP) tramite **estrazione di features** da immagini MRI e **classificazione** attraverso algoritmi di Machine Learning.

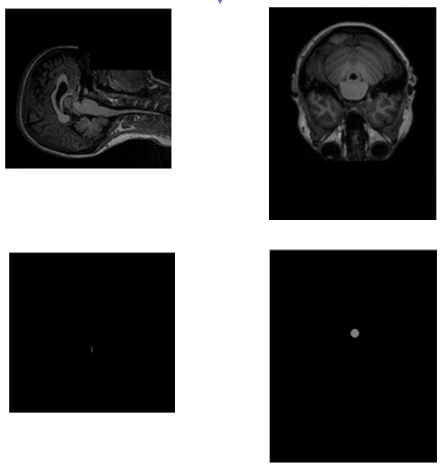
TASK 1

Estrazione features radiomiche



Matlab

Importiamo e ruotiamo img e roi



Utilizziamo l'algoritmo **SERA** per estrarre le caratteristiche dalla ROI dell'immagine

Salvataggio delle features estratte

Estrazione features con SERA

```
65  cd(directory)
66  clearvars -except FullFeatures F
67  tic
68  addpath 'C:\Users\HP\Desktop\Task1\texture_code' % Radiomics features calcualt
69
70  DataDir = 'C:\Users\HP\Desktop\Task1\paziente';
71  dbsName = 'IBSI_MR';
72
73  try
74      VoxelSizeInfo = xlsread('C:\Users\HP\Desktop\Task1\voxel_info.xls');
```

```
269 %% Saving the results
270 time    = now;
271 str     = datestr(time,0); str(str==':')='_'; str(str==' ')=',';
272 if ifSave
273     save(['C:\Users\HP\Desktop\Task1\risultati\Results',dbsName,'__',int2str(N
274         'features_all', 'PatDirName', 'SkippedCases','isotVoxSize', 'isotVoxSi
275 end
276
```

```
277 close all
278 clear all
279
280 directory=uigetdir;   %Parto dalla directory iniziale presente sul desktop
281 cd(directory);
282
283
284 features= load("ResultsIBSI_MR__1_cases_Isotrop_1mm_1_bins_FBN_Discr___14-Dec-
285 FullFeatures=num2cell(features.features_all);
286 tabella={features.features_all };
287 writecell(tabella,['C:\Users\HP\Desktop\Task1\risultati\FeatureExtraction.xls'
288
```

```
105 Feats2out    = 1;            % Select carefully! (default 2) which set of featu
106
107 ImgFileName = 'img';      % The filename that includes the image variable. It s
108 ROIFileName = 'ROI';    % The filename that includes the ROI variable. It shoul
109                           % !!!!!! Currently, the name of variables inside I
110                           % !!!!!! Make sure to set them under section "%% V
111
112 ifSave      = 1;            % whether to save the results. Specify the target
113 qntz        = 'Uniform';    % An extra option for FBN Discretization Type: Eit
114
```

Python

Carichiamo tutte le librerie utili al progetto

Importiamo l'immagine e la maschera settando l'estensione del file e li plottiamo a video

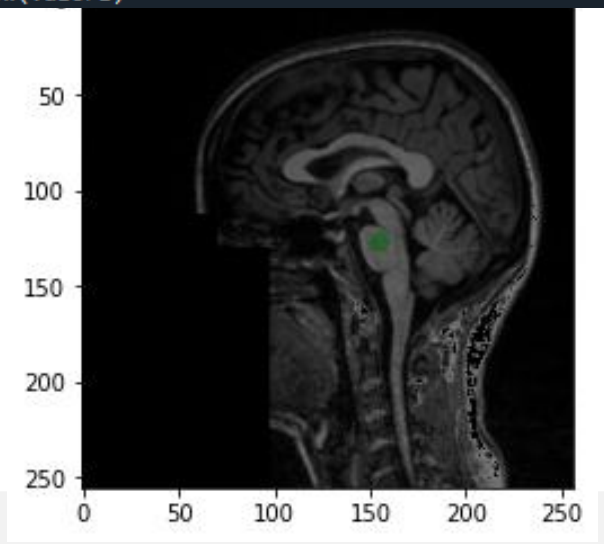
Definiamo la cartella per salvare i risultati e settiamo le impostazioni a seconda della modalità di imaging con cui stiamo lavorando

Resempling e campionamento

Facciamo un check per vedere se la maschera si sovrappone perfettamente altrimenti dovremo fare un resemple

Estrazione Features e salvataggio in file csv

```
#ESTRAZIONE DI FEATURES
#selezioniamo prima lo strumento e poi richiamiamo i setting
fex= featureextractor.RadiomicsFeatureExtractor(**setting)
fex.disableAllFeatures()
#disabilitiamo tutte le feature, abilitiamo le classi che ci interessano, in questo caso sono tutte
fex.enableFeatureClassByName("firstorder")
fex.enableFeatureClassByName("shape")
fex.enableFeatureClassByName("glcm")
fex.enableFeatureClassByName("glrlm")
fex.enableFeatureClassByName("glszm")
fex.enableFeatureClassByName("ngtdm")
fex.enableFeatureClassByName("gldm")
#

#Execute
result= fex.execute(image, mask, label=1) #label =1 sto prendendo solo la zona interessata

nome_risultati= "C://Users//pc//Desktop//Elaborazione Python-Slicer//feature_paziente.csv"
#salviamo i risultati
fieldnames = list (result)
import csv
with open(nome_risultati, mode= 'w') as csv_file:
    writer = csv.DictWriter(csv_file, fieldnames=fieldnames) #fielnames sono le features estratte
    writer.writeheader()
    result1= dict(result)
    valori= result1.values()
    csvwriter = csv.writer(csv_file)
    csvwriter.writerow(valori)
```



3D Slicer

Carichiamo l'immagine

↓

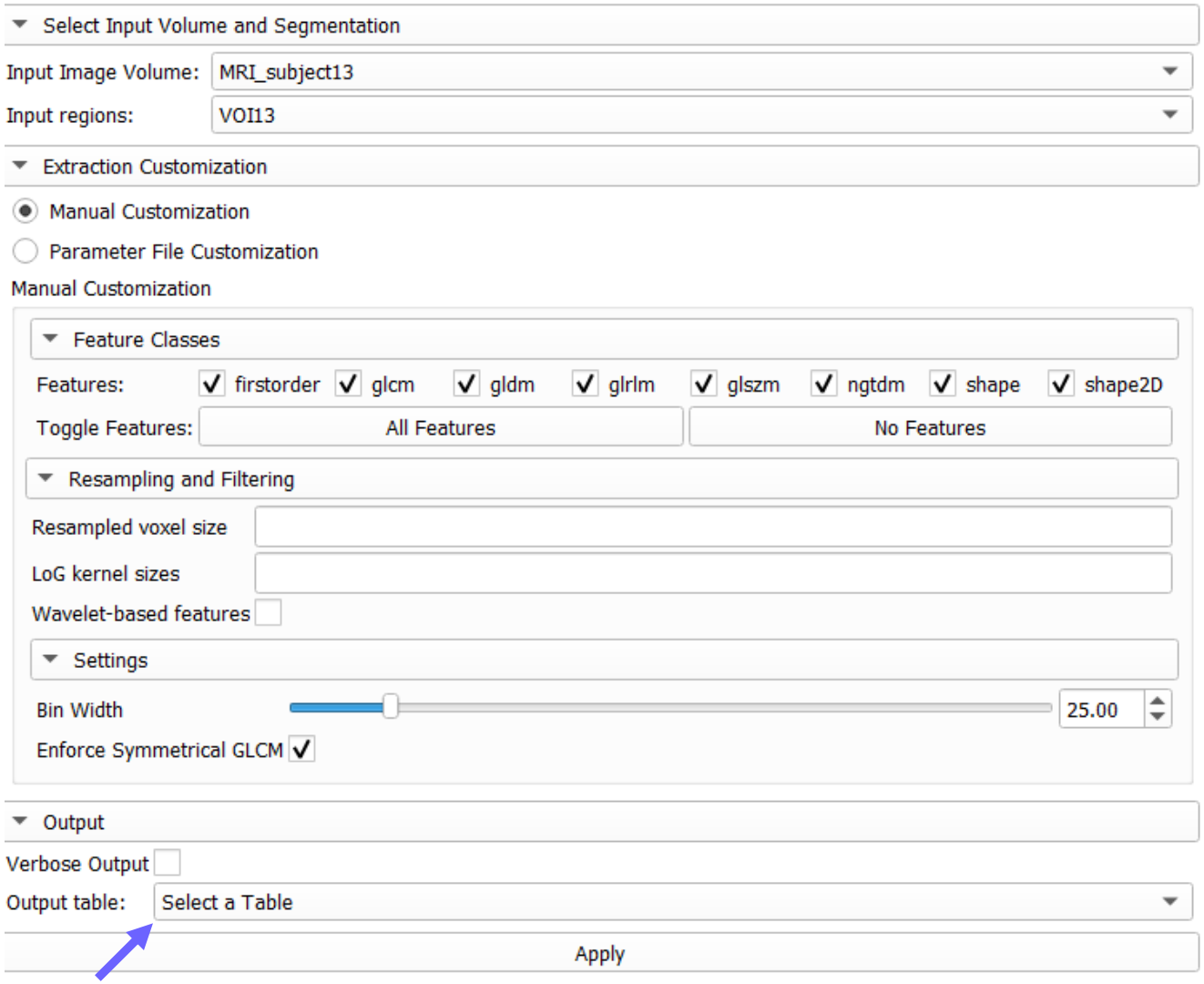
Importiamo la ROI o la VOI a seconda del tipo di immagine su cui stiamo lavorando (2D o 3D)

↓

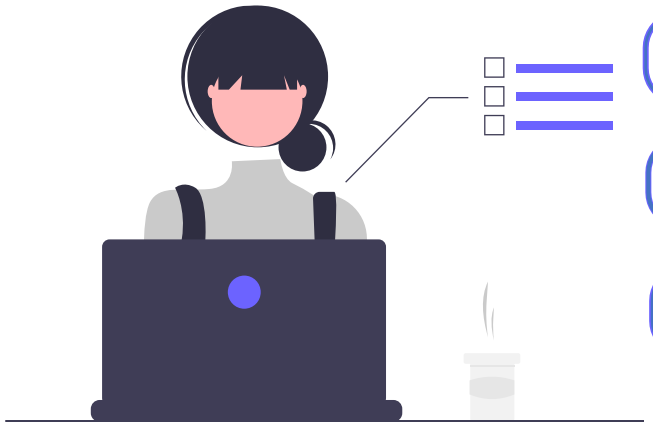
Tramite Radiomics estraiamo le features

↓

Infine le salviamo in formato tabellare



Confronto features estratte da Matlab e 3DSlicer



- Features
- Shape
- Statistic
- Histogram
- GLCM
- GLRLM
- GLSZM
- NGTDM
- NGLDM

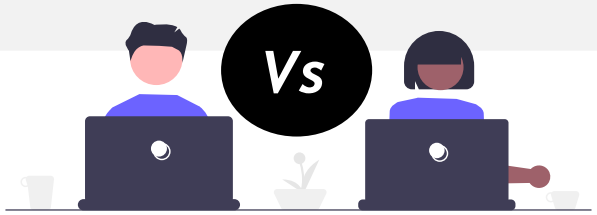
features_nam	results matlab(im_ruota	results matlab(im_non_ruota	results sli
Joint maximum	0,121587604	0,048484847	
Joint average	4,487050057	1,062215924	2,63957034
Joint variance	2,967048407	0,526394784	
Joint entropy	4,963249683	0,77316004	3,134749763
Difference average	1,274212599	0,3528409	0,817325017
Difference variance	1,604516149	0,384215593	0,463415711
Difference entropy	2,071660519	0,408924162	1,206684331
Sum average	8,974100113	2,124431849	5,279140679
Sum variance	8,568087578	1,151601911	
Sum entropy	3,423364639	0,53079164	2,082255066
Angular second moment	0,043198939	0,037735164	
Contrast	3,300105572	0,953977287	1,145530146
Dissimilarity	1,274212599	0,3528409	

features_nam	results matlab(im_ruota	results matlab(im_non_ruota	results sli
Coarseness	0,09484724	0,511688292	0,069200732
Contrast	0,107022598	0,319067448	0,057784554

features_nam	results matlab(im_ruota	results matlab(im_non_ruota	results sli
Low dependence emphasis	0,272877276	0,701641262	
High dependence emphasis	11,79487133	2,283221722	
Low grey level count emphasis	0,120843925	0,118592255	
High grey level count emphasis	22,79487228	20,53495979	
pendence low grey level emphasis	0,026247712	0,085102916	
ependence high grey level emphasis	6,371737957	14,6285944	
pendence low grey level emphasis	1,620194912	0,258161247	
ependence high grey level emphasis	249,0256348	45,59700775	
Grey level non-uniformity	13,1282053	2,060737133	
y level non-uniformity normalised	0,168310329	0,279725164	
Dependence count non-uniformity	16,53846169	4,738423348	
count non-uniformity normalised	0,212031558	0,635093093	
Dependence count percentage	1	1	
Grey level variance	3,117685795	2,142830133	
Dependence count variance	2,168967724	0,209389478	
Dependence count entropy	4,605995655	2,266743422	
Dependence count energy	0,04536489	0,234794676	
Run length variance	0,375910401	0,054560419	0,525320718
Run entropy	3,692546606	2,066884041	2,915382187
Intensity at 10% volume	2	2	

features_nam	results matlab(im_ruota	results matlab(im_non_ruota	results sli
Small zone emphasis	0,575584769	0,793749988	0,514206752
Large zone emphasis	18,07407379	1,963528156	41,9047619
Low grey level emphasis	0,118733622	0,12011186	0,314074074
High grey level emphasis	22,66666603	21,04491425	7,904761905
mall zone low grey level emphasis	0,056084346	0,098067284	0,171993373
nall zone high grey level emphasis	13,18457699	16,73963738	4,24536967
arge zone low grey level emphasis	1,680917621	0,216948196	8,385171958
rge zone high grey level emphasis	416,5185242	40,48246765	341,1428571
Grey level non-uniformity	4,481481552	1,506421328	5,095238095
y level non uniformity normalised	0,165980801	0,264426023	0,242630385
Zone size non-uniformity	8,70370388	4,045094013	5,476190476
e size non-uniformity normalised	0,322359383	0,6739434	0,260770975
Zone percentage	0,346153855	0,792863727	0,233333333
Grey level variance	3,566529512	2,269925594	1,292517007
Zone size variance	9,728395462	0,218241677	23,53741497
Zone size entropy	3,958229065	2,240544796	3,820888851

Matlab



PyRadiomics

GLSZM: *quantificano la grey level zones, che rappresenta il numero di voxel in regioni che condividono la stessa intensità*

3.8.16 Zone size entropy

Let $p_{ij} = s_{ij}/N_s$. *Zone size entropy* is then defined as:

$$F_{szm.zs.entr} = -\sum_{i=1}^{N_g}\sum_{j=1}^{N_z} p_{ij}\log_2 p_{ij}$$

10. Zone Entropy (ZE)

$$ZE = -\sum_{i=1}^{N_g}\sum_{j=1}^{N_s} p(i,j)\log_2(p(i,j)+\epsilon)$$

Here, ϵ is an arbitrarily small positive number ($\approx 2.2 \times 10^{-16}$).

ZE measures the uncertainty/randomness in the distribution of zone sizes and gray levels. A higher value indicates more heterogeneneity in the texture patterns.

GLRLM: *quantificano la lunghezza dei gray level run, ovvero delle sequenze contigue di voxel con lo stesso livello di grigio*

3.7.16 Run entropy HJ90

Run entropy was investigated by Albregtsen et al.[3]. Again, let $p_{ij} = r_{ij}/N_s$. The entropy is then defined as:

$$F_{rlm.rl.entr} = -\sum_{i=1}^{N_g}\sum_{j=1}^{N_r} p_{ij}\log_2 p_{ij}$$

10. Run Entropy (RE)

$$RE = -\sum_{i=1}^{N_g}\sum_{j=1}^{N_r} p(i,j|\theta)\log_2(p(i,j|\theta)+\epsilon)$$

Here, ϵ is an arbitrarily small positive number ($\approx 2.2 \times 10^{-16}$).

Confronto features estratte da Python e 3DSlicer

Entrambi gli ambienti utilizzano il pacchetto **Pyradiomics**

107 features estratte:

- Shape
- First Order
- Glcm
- Glrlm
- Glszm
- Ngtdm
- Gldm

glcm	ngtdm	Busyness	3,326	3,326
glcm	ngtdm	Coarseness	0,011	0,011
glcm	ngtdm	Complexity	13,058	13,058
glcm	ngtdm	Contrast	0,034	0,034
glcm	ngtdm	Strength	0,173	0,173
glcm	gldm	DependenceEntropy	5,580	5,580
glcm	gldm	DependenceNonUniformity	47,178	47,178
glcm	gldm	DependenceNonUniformityNormalized	0,076	0,076
glcm	gldm	DependenceVariance	13,300	13,300
glcm	gldm	GrayLevelNonUniformity	170,519	170,519
glcm	gldm	GrayLevelVariance	1,057	1,057
glcm	gldm	HighGrayLevelEmphasis	15,165	15,165
glcm	gldm	LargeDependenceEmphasis	76,913	76,913
glcm	gldm	LargeDependenceHighGrayLevelEmph	1145,375	1145,375
glcm	gldm	LargeDependenceLowGrayLevelEmpha	6,157	6,157
glcm	gldm	LowGrayLevelEmphasis	0,098	0,098
glcm	gldm	SmallDependenceEmphasis	0,053	0,053
glcm	gldm	SmallDependenceHighGrayLevelEmpha	0,923	0,923
glcm	gldm	SmallDependenceLowGrayLevelEmpha	0,010	0,010
glcm	glszm	HighGrayLevelZoneEmphasis	16,679	16,679
glcm	glszm	LargeAreaEmphasis	3620,607	3620,607
glcm	glszm	LargeAreaHighGrayLevelEmphasis	51972,179	51972,179
glcm	glszm	LargeAreaLowGrayLevelEmphasis	288,570	288,570
glcm	glszm	LowGrayLevelZoneEmphasis	0,254	0,254
glcm	glszm	SizeZoneNonUniformity	7,000	7,000
glcm	glszm	SizeZoneNonUniformityNormalized	0,250	0,250
firstorc	glszm	SmallAreaEmphasis	0,473	0,473
firstorc	glszm	SmallAreaHighGrayLevelEmphasis	9,055	9,055
firstorc	glszm	SmallAreaLowGrayLevelEmphasis	0,119	0,119
firstorc	glszm	ZoneEntropy	3,839	3,839
firstorc	glszm	ZonePercentage	0,045	0,045
firstorder	glszm	ZoneVariance	3131,881	3131,881
firstorder	Skewness		0,122	0,122
firstorder	TotalEnergy		102318840,793	102318840,793
firstorder	Uniformity		0,275	0,275
firstorder	Variance		587,565	587,565

TASK 2

Classificazione mediante algoritmi di ML

Machine Learning Workflow

Dataset

Preparazione dei dati

Selezione algoritmo

Training del modello

Test del modello

26 pazienti x 107 features

14 PSP-RS

12 vPSP

Classification Learner App

Implementazione
in Matlab di

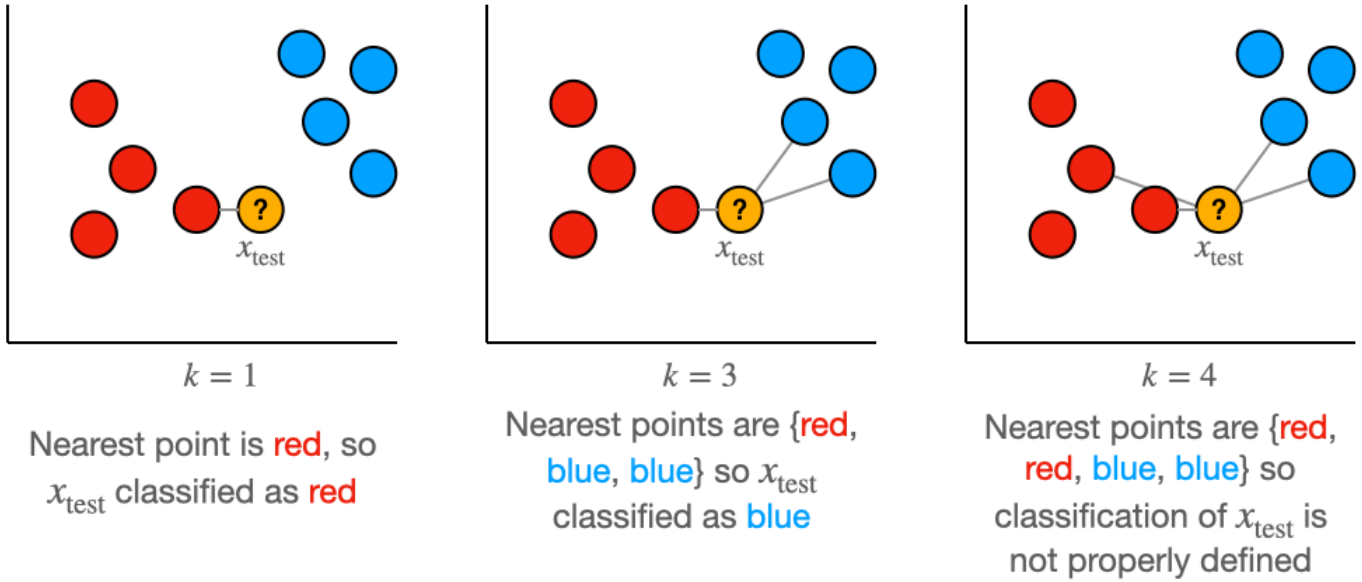
- Cosine KNN
- Weighted KNN

Algoritmo K-Nearest Neighbors

Classificatore di apprendimento **supervisionato non parametrico**

Utilizzato nel riconoscimento di pattern per la classificazione di oggetti basandosi sulle caratteristiche degli **oggetti vicini** a quello considerato.

Il valore k nell'algoritmo kNN definisce quanti vicini verranno controllati per determinare la classificazione



Cosine KNN

La distanza metrica tra le osservazioni (trattate come vettori) è calcolata come 1 meno il coseno dell'angolo compreso

Weighted KNN

Viene dato più peso ai punti che si trovano nelle vicinanze e meno peso ai punti che sono più lontani

Implementazione dei modelli KNN in MatLab

Cosine KNN

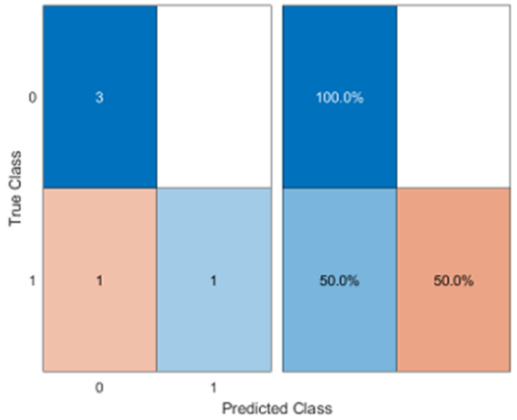
```
            % Implementazione KNN model Cosine
            knnmodel = fitcknn(data_train, malattia_pazienti_training,'NSMethod','exhaustive','Distance','cosine', 'NumNeighbors', NN);
            predClass = predict(knnmodel, data_test);
            iscorrect = predClass == malattia_pazienti_test; % classe predetta CORRISPONDE con la classe vera
            accuracy = sum(iscorrect)/numel(predClass); % (num di classi predette correttamente)/(totale delle classi predette)
            iswrong = predClass ~= malattia_pazienti_test; % classe predetta NON CORRISPONDE con la classe vera
            misclassrate = sum(iswrong)/numel(predClass); % (num di classi predette NON correttamente)/(totale delle classi predette)

            sum_accuracy = sum_accuracy + accuracy;
        end

        accuracy_media = sum_accuracy/k;

        if accuracy_media > accuracy_max
            accuracy_max = accuracy_media;
            NN_accuracy_max = NN;
        end

    end

    final=['La miglior accuracy ( ', num2str(accuracy_max),' ) si ha con NumNeighbors pari a ', num2str(NN_accuracy_max)];
    disp(final)

    if accuracy_max > best_accuracy
        best_accuracy = accuracy_max;
        k_best_accuracy = k;
    end
end

disp('_____________________________________________________________________________')
Z=['BEST ACCURACY = ', num2str(best_accuracy), ' , K = ', num2str(k_best_accuracy)];
disp(Z);
```

Risultati:

Cosine KNN

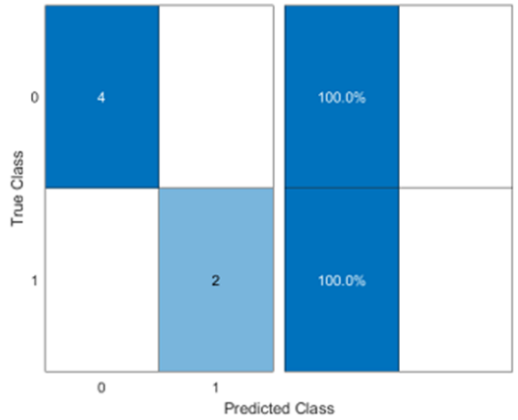
```
_____________________________
Set diviso in K = 3 gruppi
La miglior accuracy ( 0.76852 ) si ha con NumNeighbors pari a 3

_____________________________
Set diviso in K = 4 gruppi
La miglior accuracy ( 0.85119 ) si ha con NumNeighbors pari a 4

_____________________________
Set diviso in K = 5 gruppi
La miglior accuracy ( 0.88 ) si ha con NumNeighbors pari a 5

_____________________________
Set diviso in K = 6 gruppi
La miglior accuracy ( 0.84167 ) si ha con NumNeighbors pari a 5

_____________________________
Set diviso in K = 7 gruppi
La miglior accuracy ( 0.85714 ) si ha con NumNeighbors pari a 5

_____________________________________________________________________
BEST ACCURACY = 0.88 , K = 5
```

----- Fold 1 -----



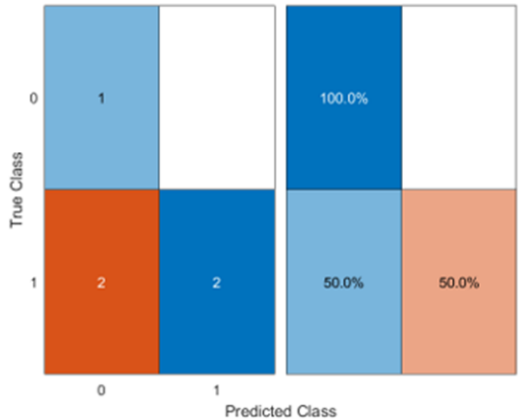
Numero di Veri Positivi (PSP_RS correttamente predetti): 3
Numero di Falsi Positivi (vPSP predetti come PSP_RS): 1
Numero di Veri Negativi (vPSP correttamente predetti): 1
Numero di Falsi Negativi (PSP_RS predetti come vPSP): 0

----- Fold 2 -----



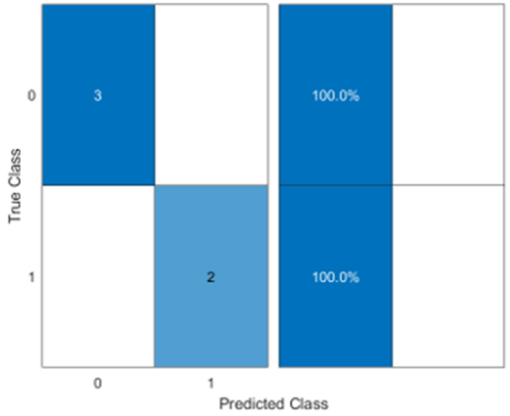
Numero di Veri Positivi (PSP_RS correttamente predetti): 4
Numero di Falsi Positivi (vPSP predetti come PSP_RS): 0
Numero di Veri Negativi (vPSP correttamente predetti): 2
Numero di Falsi Negativi (PSP_RS predetti come vPSP): 0

----- Fold 3 -----



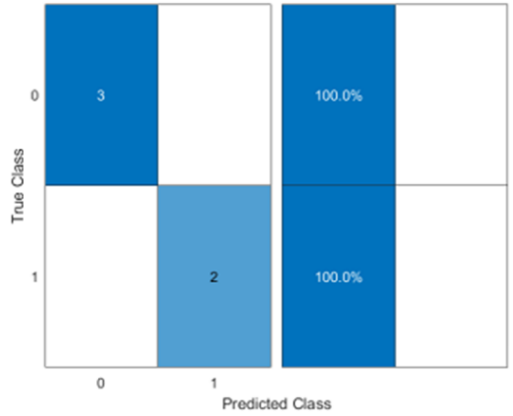
Numero di Veri Positivi (PSP_RS correttamente predetti): 1
Numero di Falsi Positivi (vPSP predetti come PSP_RS): 2
Numero di Veri Negativi (vPSP correttamente predetti): 2
Numero di Falsi Negativi (PSP_RS predetti come vPSP): 0

----- Fold 4 -----



Numero di Veri Positivi (PSP_RS correttamente predetti): 3
Numero di Falsi Positivi (vPSP predetti come PSP_RS): 0
Numero di Veri Negativi (vPSP correttamente predetti): 2
Numero di Falsi Negativi (PSP_RS predetti come vPSP): 0

----- Fold 5 -----



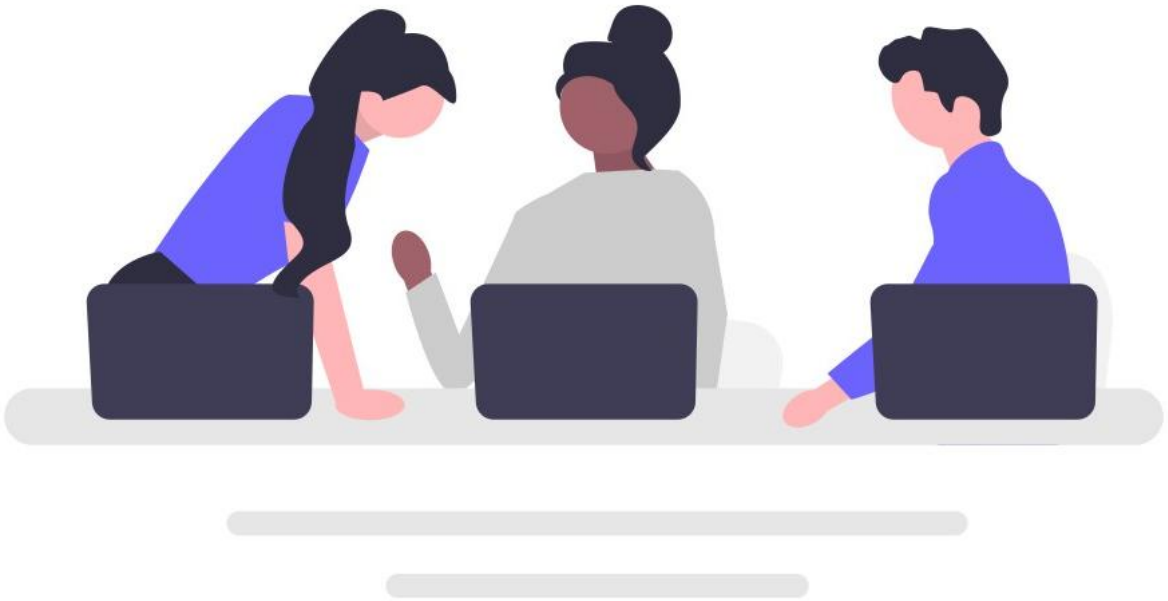
Numero di Veri Positivi (PSP_RS correttamente predetti): 3
Numero di Falsi Positivi (vPSP predetti come PSP_RS): 0
Numero di Veri Negativi (vPSP correttamente predetti): 2
Numero di Falsi Negativi (PSP_RS predetti come vPSP): 0

Numero di fold = 5
Numero di vicini più vicini = 5

Weighted KNN

```
knnmodel = fitcknn(data_train, malattia_pazienti_training,'DistanceWeight',"squaredinverse",'NumNeighbors', NN);
```

```
Set diviso in K = 3 gruppi
La miglior accuracy ( 0.87963 ) si ha con NumNeighbors pari a 2

_____________________________
Set diviso in K = 4 gruppi
La miglior accuracy ( 0.85119 ) si ha con NumNeighbors pari a 5

_____________________________
Set diviso in K = 5 gruppi
La miglior accuracy ( 0.92 ) si ha con NumNeighbors pari a 5

_____________________________
Set diviso in K = 6 gruppi
La miglior accuracy ( 0.88333 ) si ha con NumNeighbors pari a 2

_____________________________
Set diviso in K = 7 gruppi
La miglior accuracy ( 0.89286 ) si ha con NumNeighbors pari a 2

_____________________________________________________________________________
BEST ACCURACY = 0.92 , K = 5
```



Grazie per l’attenzione!